

Remote Volume Control (RVCW)

Relatório de Avaliação de Segurança

Estudo de Caso

Arquitetura · Segurança · Modelagem de ameaças · Recomendações

Data de análise: Maio de 2026

Conteúdos

1. Informação Geral	3
1.1 Visão Geral.....	3
1.2 Histórico de Versão	3
1.3 Objetivo da Avaliação	3
1.4 Escopo	3
2. Sumário Executivo	5
3. Relatório de Análise da Arquitetura.....	7
3.1 Estrutura de Módulos e Pacotes	7
3.2 Componentes Chave	7
3.3 Configuração de Compilação.....	8
4. Explicação do ciclo de vida do Android	9
4.1 Ciclo de vida de MainActivity	9
4.2 Ciclo de vida de WebServerService.....	9
4.3 Ciclo de vida de BootCompletedReceiver	9
4.4 Threading Model.....	10
5. Análise do Servidor Ktor.....	11
5.1 Configuração do Servidor	11
5.2 Plugins Instalados	11
5.3 Rotas.....	11
5.4 Mecanismo de Autenticação.....	12
6. Revisão de Segurança.....	13
6.1 Autenticação & Autorização.....	13
6.2 Segurança nos Transportes.....	13
6.3 Validação de Input.....	13
6.4 Permissões Android	14
6.5 Componentes Exportados	14
6.6 Armazenamento de Dados	14
7. Modelagem de Ameaças.....	15
7.1 Análise STRIDE.....	15
7.2 Cenários de Ataque	15
Cenário A — Captura Passiva de Dados	15
Cenário B — Chave Padrão.....	15
Cenário D — Negação de Serviço	16
8. Análise de Risco	17

9. Recomendações	18
Prioridade 1 — Crítica (corrigir antes da distribuição para produção)	18
R3: Remover ou rotacionar a chave de autenticação padrão	18
Prioridade 2 — Alta	18
R4: Adicionar limitação de taxa (rate limiting) em falhas de autenticação	18
R5: Vincular apenas à interface WiFi	18
R6: Excluir chave de autenticação do backup do Android	18
Prioridade 3 — Média	19
R7: Sanitizar respostas de StatusPages	19
R8: Adicionar Configuração CORS	19
R9: Remover a permissão CHANGE_NETWORK_STATE	19
R10: Corrigir bug de registro do BroadcastReceiver	19
Priority 4 — Baixa	19
R11: Habilitar ProGuard/R8 em builds de lançamento	19
R12: Adicionar validação da força da chave de autenticação	19
R13: Expandir a cobertura de testes	19
10. Lições Aprendidas	21

1. Informação Geral

O Remote Volume Control é um aplicativo Android desenvolvido como um projeto pessoal que permite o controle remoto do volume de áudio de um dispositivo Android por meio de uma interface web hospedada diretamente no dispositivo.

O aplicativo incorpora um servidor HTTP Ktor que escuta solicitações de dispositivos conectados à mesma rede local. Usuários autorizados podem aumentar, diminuir, silenciar ou ajustar o volume do dispositivo por meio de uma interface baseada em navegador.

O projeto foi originalmente desenvolvido com foco na funcionalidade e facilidade de uso em ambientes de rede doméstica confiáveis, em vez de reforço de segurança.

1.1 Visão Geral

Repositório: `github.com/RiannReis/remote-volume-control`

Projeto: Remote Volume Control (RVCW)

Autor: Riann dos Reis Silva

Versão: 1.0

1.2 Histórico de Versão

V0.1: [02/05/2026] - Primeira versão do relatório

V0.2: [16/05/2026] - Adicionados detalhes da arquitetura

V0.3: [20/05/2026] - Adicionada revisão de segurança e modelagem de ameaças

V0.4: [24/05/2026] - Adicionadas recomendações

V0.5: [28/05/2026] - Imagens adicionadas

V1.0: [08/06/2026] - Versão final

1.3 Objetivo da Avaliação

O objetivo desta avaliação é analisar o nível de segurança da aplicação por meio de revisão arquitetônica, modelagem de ameaças, análise da superfície de ataque e validação manual de possíveis vulnerabilidades de segurança.

A avaliação concentra-se na identificação de riscos de segurança associados a:

- Mecanismos de autenticação
- Comunicação em rede
- Exposição de dados
- Disponibilidade do serviço
- Considerações de segurança específicas do Android

1.4 Escopo

Dentro do Escopo

- Pacote de aplicativo Android (APK)
- Servidor HTTP Ktor embutido
- Mecanismo de autenticação

- Interface web
- Componentes de serviço Android
- Comunicação em rede local
-

Fora do escopo

- Sistema operacional Android
- Infraestrutura de terceiros
- Ataques de acesso físico
- Cenários com dispositivos root
- Ataques de rede externa

2. Sumário Executivo

Este relatório apresenta os resultados de uma avaliação de segurança realizada no aplicativo Android Remote Volume Control (RVCW), um projeto pessoal desenvolvido para fornecer controle remoto de áudio baseado em LAN por meio de um servidor web Ktor embutido.

Originalmente desenvolvido com foco em funcionalidade e facilidade de uso, o aplicativo foi posteriormente revisitado como um estudo de caso prático de Segurança de Aplicativos (AppSec). A avaliação combinou revisão arquitetônica, modelagem de ameaças (STRIDE), análise de superfície de ataque, revisão estática e validação manual de possíveis vulnerabilidades de segurança.

Do ponto de vista da engenharia de software, o aplicativo demonstra uma arquitetura estruturada e segue diversas práticas de desenvolvimento para Android. A base de código apresenta uma separação de responsabilidades entre a interface do usuário, os serviços em segundo plano e os componentes do servidor HTTP. A implementação faz uso dos serviços em primeiro plano do Android, persistência de SharedPreferences, gerenciamento de permissões, suporte à localização e design modular baseado em Kotlin.

A revisão da arquitetura identificou diversas características, incluindo:

- Separação entre as camadas de interface do usuário, serviço e rede.
- Gerenciamento do ciclo de vida do Android.
- Uso consistente de Kotlin e APIs da plataforma Android.
- Implementação adequada da funcionalidade de controle de volume por meio do Android AudioManager.
- Definições de rotas HTTP estruturadas usando o framework Ktor.

A avaliação também identificou diversas vulnerabilidades de segurança, a maioria das quais originadas de decisões arquitetônicas e de design, e não de defeitos de implementação.

As seguintes constatações foram validadas manualmente durante os testes:

- Comunicação HTTP em texto não criptografado (sem TLS).
- Segredo de autenticação transmitido por meio de parâmetros de URL.
- Vulnerabilidade de repetição causada por solicitações reutilizáveis indefinidamente.
- Ausência de mecanismos de limitação de taxa ou controle de requisições.
- Divulgação de informações por meio de mensagens de exceção retornadas pelo servidor Ktor.
- Funcionalidade de backup do Android habilitada por meio da configuração allowBackup.

Um resultado notável da avaliação foi a validação manual bem-sucedida das constatações inicialmente identificadas durante a revisão. Isso reforçou a importância de combinar a análise automatizada com testes práticos e verificação baseada em evidências.

A postura geral de segurança do aplicativo é consistente com seu propósito original como um utilitário de uso pessoal destinado à operação em redes domésticas confiáveis. No entanto, vários cenários de ataque tornam-se viáveis quando um invasor obtém acesso ao mesmo

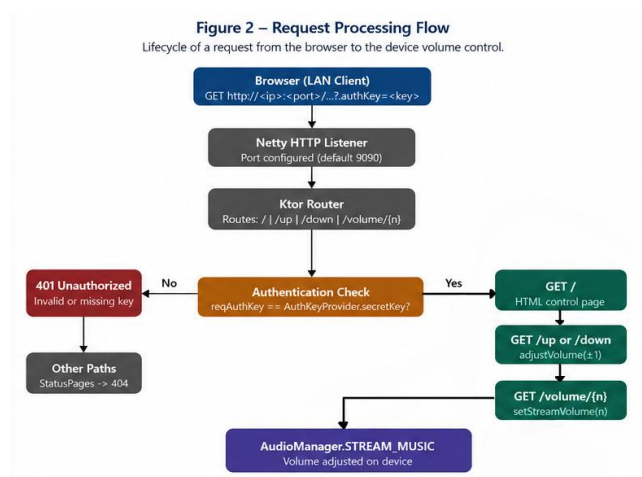
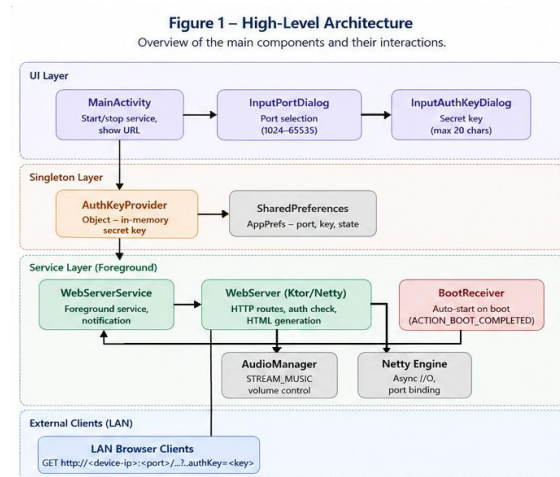
segmento de rede, principalmente devido à falta de criptografia de transporte, proteção contra repetição e controles de limitação de solicitações.

Não foram identificadas evidências de vulnerabilidades de execução remota de código ou escalonamento de privilégios. Contudo, constatou-se que o projeto de autenticação é suscetível a ataques de adivinhação de credenciais quando chaves de autenticação fracas são utilizadas, devido à ausência de mecanismos de limitação de taxa ou bloqueio de conta. Mesmo assim, as descobertas demonstram vários problemas comuns de segurança de aplicativos (AppSec) frequentemente encontrados em aplicativos desenvolvidos internamente e projetos pessoais.

Esta avaliação destaca o valor da aplicação de técnicas de modelagem de ameaças, revisão de arquitetura, análise de superfície de ataque e validação manual a projetos de software existentes. Além de identificar descobertas específicas, o exercício proporcionou uma oportunidade prática de avaliar um aplicativo do mundo real por meio de uma perspectiva focada em segurança e documentar os riscos e observações resultantes.

3. Relatório de Análise da Arquitetura

O Remote Volume Control (RVCW) é um aplicativo Android de módulo único que expõe um servidor HTTP embutido na rede local, permitindo que qualquer navegador na mesma rede Wi-Fi ajuste o volume de mídia do dispositivo. O código-fonte foi escrito em Kotlin para as APIs do Android 24 a 34 e utiliza o Ktor 3.0.0 com o mecanismo Netty para servir o servidor HTTP.



3.1 Estrutura de Módulos e Pacotes

O aplicativo está organizado em um módulo Gradle (:app) com a seguinte hierarquia de pacotes:

- com.riannreis.rvcw — raiz: Singleton AuthKeyProvider
- com.riannreis.rvcw.main — MainActivity (ponto de entrada da interface do usuário)
- com.riannreis.rvcw.dialogs — InputPortDialogFragment, InputAuthKeyDialogFragment
- com.riannreis.rvcw.server — WebServer (lógica do Ktor), WebServerService (serviço em primeiro plano)
- com.riannreis.rvcw.receiver — BootCompletedReceiver (inicialização automática)

3.2 Componentes Chave

Componente	Tipo	Responsabilidade
MainActivity	AppCompatActivity	UI Principal: iniciar/parar o serviço, exibir URL, dialogs para a configuração de porta/chave.
WebServerService	Foreground Service	Mantém o servidor Ktor/Netty vivo; exibe notificações; recebe reinicializações START_STICKY.

WebServer	Classe simples	Define as rotas Ktor: GET /, /up, /down, /volume/{n}. Valida o parâmetro de consulta authKey por requisição. Chama AudioManager.
AuthKeyProvider	Kotlin object (singleton)	Armazenamento em memória da chave secreta atual. Padrão: "3x@mplE_S3crEt_K3y!"
BootCompletedReceiver	BroadcastReceiver	Escuta ACTION_BOOT_COMPLETED; lê autoStartEnabled de SharedPreferences e inicia o serviço se definido.
InputPortDialogFragment	DialogFragment	Valida a porta no intervalo de 1024 a 65535; Persiste em SharedPreferences "AppPrefs".
InputAuthKeyDialogFragment	DialogFragment	Valida chave ≤ 20 caracteres; persiste em SharedPreferences "AppPrefs".

3.3 Configuração de Compilação

- compileSdk / targetSdk: 34 (Android 14).
- minSdk: 24 (Android 7.0 ~97% de cobertura de dispositivos).
- Versão do Ktor: 3.0.0 (server-core, server-netty, server-html-builder, server-status-pages).
- Ferramentas de compilação: Gradle 8.7, AGP 8.6.0, Kotlin 1.9.0, JVM target 1.8.
- isMinifyEnabled = false na versão de lançamento, ProGuard/R8 não aplicado.
- Empacotamento: Conflitos de META-INF do Netty excluídos por meio de packagingOptions.

4. Explicação do ciclo de vida do Android

4.1 Ciclo de vida de MainActivity

A MainActivity segue os callbacks padrão da AppCompatActivity. Comportamento principal em cada etapa:

Callback	O que RVCW faz
onCreate()	Carrega o estado salvo do servidor, endereço IP, porta e chave de autenticação das SharedPreferences. Constrói o rvcwURL. Solicita a permissão POST_NOTIFICATIONS no Android 13+. Inicializa todas as referências de visualização e o ouvinte de botões.
onStart()	Chama updateActivity() para sincronizar a interface do usuário com o sinalizador isRunning. Na API 33+, registra o BroadcastReceiver do ServerStateReceiver (RECEIVER_NOT_EXPORTED) para escutar as transmissões SERVER_STATE_UPDATE do serviço.
onStop()	Cancela o registro do serverStateReceiver. NOTA: em APIs < 33, o receptor nunca é registrado (a condição verifica se API >= 0 e depois >= TIRAMISU), portanto, unregisterReceiver() lançará uma exceção se for chamado sem um registro prévio — um bug latente nas APIs 26–32.
onSaveInstanceState()	O valor booleano “isRunning” é mantido no Bundle para que o estado da interface do usuário sobreviva à rotação. O próprio WebServerService não é afetado por alterações de configuração.

4.2 Ciclo de vida de WebServerService

O serviço é declarado como um serviço em primeiro plano com foregroundServiceType="mediaPlayback".

Callback	Comportamento
onCreate()	Cria um NotificationChannel (API 26+), obtém o serviço de sistema AudioManager e instancia um WebServer.
onStartCommand()	Lê PORT_VALUE e AUTH_KEY dos extras da Intent. Chama startForeground() com uma notificação persistente e, em seguida, webServer.startServer(). Retorna START_STICKY — o sistema operacional reiniciará o serviço após o encerramento do processo, reenviando a última Intent.
onDestroy()	Chama webServer.stopServer() que chama server.stop(1000, 10000) — um desligamento correto com períodos de tolerância de 1 s para conexão e 10 s para desconexão.
onBind()	Retorna null — o serviço não está vinculado; a comunicação de volta para a atividade usa LocalBroadcasts (SERVER_STATE_UPDATE).

4.3 Ciclo de vida de BootCompletedReceiver

O Receiver é ativado brevemente em onReceive(), lê as SharedPreferences e inicia condicionalmente o serviço em primeiro plano através de startForegroundService(). Ele termina

prontamente, como exigido para BroadcastReceivers (que possuem uma janela ANR de 10 segundos). Nenhum trabalho assíncrono é executado no próprio receptor.

4.4 Threading Model

O aplicativo utiliza três contextos de threading distintos:

- Thread principal (interface do usuário) — Todos os callbacks de Activity/Fragment, manipuladores de cliques de botões, leituras/gravações de SharedPreferences (via apply(), que é assíncrono).
- Pool de threads Ktor/Netty — Cada requisição HTTP recebida é tratada em uma thread de trabalho Netty de um pool de threads interno. O DSL de roteamento do Ktor suspende a execução, mas as chamadas reais do AudioManager (adjustVolume, setStreamVolume) são feitas a partir desses contextos de corrotina sem despacho explícito para a thread principal.
- Despachante de corrotina IO/Padrão — embeddedServer() é iniciado com wait=false, o que significa que a inicialização do Netty não bloqueia a execução do serviço. O framework Ktor despacha internamente para suas próprias threads de mecanismo.

RISCO: O AudioManager geralmente é thread-safe para operações de volume, mas isso não está documentado como uma garantia de thread-safety. O código depende de um detalhe de implementação.

5. Análise do Servidor Ktor

5.1 Configuração do Servidor

O servidor embutido é inicializado com o Ktor 3.0.0 usando o mecanismo Netty:

```
server = embeddedServer(Netty, port = portValue) { ... }
```

Características principais:

- Mecanismo: Netty — de nível de produção, E/S assíncrona, baseado em NIO. Adequado para uso embarcado.
- Vinculação de host: Padrão (0.0.0.0) — vincula-se a TODAS as interfaces de rede, incluindo Wi-Fi, compartilhamento de internet via USB e adaptadores VPN.
- Somente HTTP — sem configuração TLS. Todo o tráfego é em texto não criptografado.
- `start(wait = false)` — inicialização não bloqueante; exceções são capturadas, mas apenas registradas.
- Tempo limite de parada: `stop(1000, 10000)` — 1 s para parar de aceitar, 10 s para solicitações em andamento.

5.2 Plugins Instalados

Plugin	Configuração
StatusPages	Captura exceções Throwable; responde com 500 + <code>cause.toString()</code> . Vaza nomes de classes de exceção internas e detalhes da pilha de chamadas para os clientes.
HTML Builder	Gera o HTML da página de controle através da DSL <code>kotlinx.html</code> . CSS e JavaScript são incorporados na resposta.

5.3 Rotas

Rota	Auth	Comportamento
GET /	Obrigatória	Retorna uma página HTML com o controle deslizante de volume atual (STREAM_MUSIC), botões para cima/para baixo e JavaScript embutido que chama novamente <code>/up</code> , <code>/down</code> , <code>/volume/{n}</code> via XHR.
GET /up	Obrigatória	Chama <code>audioManager.adjustVolume(ADJUST_RAISE, FLAG_PLAY_SOUND)</code> . Retorna o texto simples "Aumentar volume".
GET /down	Obrigatória	Chama <code>audioManager.adjustVolume(ADJUST_LOWER, FLAG_PLAY_SOUND)</code> . Retorna o texto simples "Volume abaixado".
GET /volume/{n}	Obrigatória	Analisa o segmento do caminho como um inteiro; limita-se ao intervalo <code>[0, maxVolume]</code> ; chama

		setStreamVolume(STREAM_MUSIC, n, FLAG_PLAY_SOUND). Retorna texto simples.
Qualquer outro caminho	—	O StatusPages falha; o Ktor retorna 404 por padrão. Não é tratado explicitamente no código.

5.4 Mecanismo de Autenticação

A autenticação é implementada como uma simples verificação de igualdade de strings no parâmetro de consulta `authKey` em cada manipulador de rota:

```
val reqAuthKey = call.parameters["authKey"]
if (reqAuthKey == this@WebServer.authKey) { ... }
```

Esta é uma autenticação por string de consulta — não é autenticação básica HTTP, nem tokens Bearer, nem cookies de sessão. A chave é visível na URL, nos logs do servidor e no histórico do navegador. Não há nonce, proteção contra repetição, limitação de taxa ou comparação segura em termos de tempo (embora a igualdade de strings na JVM seja efetivamente de tempo constante para strings de mesmo comprimento devido a diferenças de otimização JIT — isso não é garantido).

6. Revisão de Segurança

6.1 Autenticação & Autorização

- Segredo na string de consulta: A chave de autenticação (authKey) é transmitida na URL para cada requisição. Qualquer observador de rede na LAN pode capturá-la a partir de um único pacote. O histórico do navegador, os logs de proxy e os logs de acesso HTTP também a registram.
- Sem expiração/rotação: A chave persiste até que o usuário a altere manualmente. Não há tempo limite de sessão.
- Chave padrão fornecida: A chave padrão fixa "3x@mplE_S3crEt_K3y!" é visível publicamente no repositório, no arquivo strings.xml e no BootCompletedReceiver. Usuários que nunca alteram a chave são facilmente exploráveis.
- Sem limitação de taxa: Um atacante pode enviar tentativas de autenticação ilimitadas. Um espaço de chave ASCII de 20 caracteres é grande, mas um ataque de dicionário direcionado (padrões comuns, padrão do repositório, chaves simples escolhidas pelo usuário) não tem defesa.
- AuthKeyProvider é um objeto Kotlin: Singleton em memória. A chave pode ser lida por qualquer código em execução no mesmo processo.

6.2 Segurança nos Transportes

- Sem TLS: Toda a comunicação é feita via HTTP. A chave de autenticação, os comandos de volume e a página HTML trafegam em texto não criptografado. A captura passiva de tráfego de rede (spoofing de ARP, modo promíscuo) captura tudo facilmente.
- Vinculação a 0.0.0.0: O servidor aceita conexões em todas as interfaces. Se o dispositivo tiver uma interface de compartilhamento de internet via USB ou um túnel VPN, estes também estarão expostos, não apenas o Wi-Fi.
- Sem cabeçalhos CORS: A página HTML inclui JavaScript que faz requisições XHR. Sem CORS, uma página em outra origem não pode ler as respostas. Mas as requisições XHR (GET /up, /down) ainda são enviadas, o que significa que uma página da web maliciosa pode acionar alterações de volume entre origens diferentes se o usuário a visitar enquanto o servidor estiver em execução.

6.3 Validação de Input

- /volume/{n}: O parâmetro de volume é analisado com toIntOrNull(). Se for nulo, um código 400 é retornado. Caso contrário, é limitado ao intervalo [0, maxVolume] — correto.
- Entrada de porta: Validada para estar no intervalo [1024, 65535] — correto.
- Entrada de chave de autenticação: Apenas validação de comprimento (≤ 20 caracteres). Sem filtragem de caracteres. Caracteres especiais de URL (por exemplo, &, =, ?) na chave quebrariam a exibição da URL e potencialmente a comparação.
- StatusPages: Capturar Throwable bruto e retornar cause.toString() expõe mensagens de exceção internas (por exemplo, java.net.BindException: Address already in use) para clientes HTTP — divulgação de informações.

6.4 Permissões Android

Permissão	Avaliação
INTERNET	Requerido para o servidor HTTP. Necessário.
ACCESS_WIFI_STATE / ACCESS_NETWORK_STATE	Utilizado para detecção de endereço IP. Necessário.
CHANGE_NETWORK_STATE	Declarado, mas nunca usado no código. Deve ser removido.
RECEIVE_BOOT_COMPLETED	Requerido para a função de inicialização automática. Necessário caso a inicialização automática seja desejada.
FOREGROUND_SERVICE	Requerido. Necessário.
FOREGROUND_SERVICE_MEDIA_PLAYBACK	Especificado no manifesto e no tipo de serviço foregroundServiceType. Necessário para o tipo declarado.
MODIFY_AUDIO_SETTINGS	Requerido para o controle de volume. Necessário.
POST_NOTIFICATIONS	Requerido para notificação de serviço em primeiro plano na API 33+. Necessário.

6.5 Componentes Exportados

- MainActivity — exportado=true com filtro de intenção LAUNCHER. Esperado e necessário.
- WebServerService — exportado=false. Correto. Não pode ser iniciado por aplicativos externos.
- BootCompletedReceiver — exportado=true com filtro de intenção BOOT_COMPLETED. Necessário para que o sistema entregue a transmissão. No entanto, isso também significa que qualquer aplicativo no dispositivo poderia potencialmente enviar uma intenção BOOT_COMPLETED falsificada para acionar a inicialização automática (embora, na prática, as transmissões do sistema sejam protegidas). Considere adicionar uma verificação de permissão.

6.6 Armazenamento de Dados

- SharedPreferences ("AppPrefs", MODE_PRIVATE): Armazena porta, chave de autenticação, se está em execução e inicialização automática ativada. MODE_PRIVATE significa que somente este aplicativo pode ler/gravar. Correto.
- Configuração de backup: os arquivos backup_rules.xml e data_extraction_rules.xml estão presentes, mas não contêm regras de inclusão/exclusão. A chave de autenticação pode ser copiada para o Google Drive (allowBackup=true no manifesto) e restaurada em um dispositivo diferente ou após uma redefinição de fábrica — vazando credenciais.

7. Modelagem de Ameaças

7.1 Análise STRIDE

STRIDE	Ameaça	Cenário	Mitigação Atual
Spoofing (Falsificação de identidade)	Falsificação de chave de autenticação	O atacante intercepta a chave do tráfego HTTP e a reproduz	None — cleartext transport
Tampering (Adulteração de dados)	Manipulação de volume	Ator não autorizado envia /up ou /down continuamente	Auth key check (bypassable without TLS)
Repudiation (Repúdio)	Sem registro de auditoria	Não é possível determinar quem alterou o volume ou quando	None
Information Disclosure (Divulgação de Informações)	Vazamento de exceção	StatusPages retorna exception.toString() incluindo nomes de classe	None
Denial of Service (Negação de Serviço)	Inundação de requisições	O atacante inunda o sistema com /up ou /down até o volume máximo e mantém o volume pressionado	None — no rate limiting
Elevation of Privilege (Elevação de Privilégios)	Abuso de MODIFY_AUDIO_SETTINGS	Controle não autorizado das configurações de áudio do dispositivo além do volume de mídia	Auth key check

7.2 Cenários de Ataque

Cenário A — Captura Passiva de Dados

Um atacante na mesma rede Wi-Fi executa o tcpdump ou o Wireshark. O usuário abre a URL (por exemplo, compartilhada via chat ou código QR). O atacante captura uma requisição HTTP GET e extrai a chave de autenticação (authKey) da string de consulta. Ele agora tem acesso permanente até que o usuário altere a chave.

Cenário B — Chave Padrão

Um atacante na mesma rede Wi-Fi executa o tcpdump ou o Wireshark. O usuário abre a URL (por exemplo, compartilhada via chat ou código QR). O atacante captura uma requisição HTTP GET e extrai a chave de autenticação (authKey) da string de consulta. Ele agora tem acesso permanente até que o usuário altere a chave.

Cenário C — Manipulação de Volume de Origem Cruzada

Um site malicioso servido pela internet detecta o IP local do usuário (via WebRTC ou temporização) e envia uma requisição XHR para `http://<ip-do-dispositivo>:9090/up?authKey=<chave-estimada>`. Sem a proteção CORS preflight no servidor,

a solicitação é enviada (embora o corpo da resposta seja bloqueado pelo CORS). As alterações de volume ainda ocorrem.

Cenário D — Negação de Serviço

Um invasor automatiza o curl para enviar 1000 solicitações por segundo para `/volume/15?authKey=...` (volume máximo). O Netty irá enfileirar e processar essas solicitações. O sistema de áudio fica sobrecarregado com chamadas de ajuste de volume. O usuário não consegue diminuir o volume mais rápido do que o invasor o aumenta.

Cenário E — Vazamento de Credenciais de Backup

O backup automático do Android (`allowBackup=true`, sem regras de exclusão) faz backup das `SharedPreferences` do `AppPrefs` no Google Drive. A chave de autenticação (`authKey`) está incluída. Se a conta do Google do usuário for comprometida ou se ele restaurar o backup em outro dispositivo, a chave será exposta.

8. Análise de Risco

Risco	Probabilidade	Impacto	Severidade	Detalhes
Chave de autenticação em texto não criptografado na URL	Alto	Alto	Crítico	A captura de um único pacote resulta em acesso permanente
Chave padrão no repositório público	Alto	Alto	Alto	Usuários que nunca configuram uma chave são instantaneamente exploráveis por qualquer pessoa que leia o arquivo README
Sem limitação de taxa na autenticação	Médio	Médio	Alto	Ataques de dicionário contra chaves curtas/simples escolhidas pelo usuário
Vincula a todas as interfaces (0.0.0.0)	Médio	Médio	Médio	Expõe o servidor em dispositivos de tethering USB, VPN e hotspots involuntariamente
allowBackup inclui authKey	Baixo	Alto	Médio	A violação da conta do Google vaza a chave
StatusPages vaza detalhes da exceção	Médio	Baixo	Baixo	Nomes de classes internas expostos a clientes HTTP
Permissão CHANGE_NETWORK_STATE não utilizada	Não aplicável	Baixo	Informação	Declaradas, mas nunca usadas; superfície de ataque desnecessária
isMinifyEnabled=false na versão	Não aplicável	Baixo	Informação	O APK é fácil de fazer engenharia reversa; o algoritmo e o gerenciamento de chaves são visíveis
unregisterReceiver() na API 26–32	Baixo	Médio	Baixo	Falha latente: onStop() é chamado quando onStart() nunca registrou o receptor nesses níveis de API

9. Recomendações

Prioridade 1 — Crítica (corrigir antes da distribuição para produção)

R1: Mova authKey para o cabeçalho HTTP ou para o corpo da requisição POST

Substitua o mecanismo de string de consulta por um cabeçalho de Autorização HTTP ou um campo no corpo da requisição POST. Isso impede que a chave apareça em URLs, no histórico do navegador e nos registros de acesso ao servidor.

```
// Ktor server side
val key = call.request.headers["X-Auth-Key"]
// Browser JS side (replace XHR GET)
xhr.setRequestHeader("X-Auth-Key", storedKey);
```

R2: Habilitar HTTPS com um certificado autoassinado

Gere um certificado TLS autoassinado na primeira inicialização e utilize o Ktor em HTTPS. O usuário deverá aceitar o aviso de segurança do navegador uma única vez, após o qual todo o tráfego será criptografado. Como alternativa, utilize um nome de host mDNS local para habilitar um certificado válido através do Let's Encrypt ACME.

```
import io.ktor.server.engine.sslConnector
sslConnector(keyStore, keyAlias, { passphrase.toCharArray() }, {
    passphrase.toCharArray() }) {
    port = portValue
}
```

R3: Remover ou rotacionar a chave de autenticação padrão

Não distribuir uma chave padrão documentada publicamente. No primeiro lançamento sem uma chave salva, gerar uma chave aleatória de 16 caracteres (SecureRandom + Base64) e exibi-la ao usuário:

```
val random = SecureRandom()
val bytes = ByteArray(12).also { random.nextBytes(it) }
val key = Base64.getUrlEncoder().withoutPadding().encodeToString(bytes)
```

Prioridade 2 — Alta

R4: Adicionar limitação de taxa (rate limiting) em falhas de autenticação

Rastrear tentativas de autenticação com falha por IP de origem usando um ConcurrentHashMap com backoff exponencial. Após 5 falhas em 60 segundos, retornar 429 Too Many Requests e bloquear por 60 segundos.

R5: Vincular apenas à interface WiFi

Em vez de vincular a 0.0.0.0, detecte o IP da interface WiFi (já feito para exibição) e vincule o Netty a esse IP específico:

```
embeddedServer(Netty, host = localIp ?: "127.0.0.1", port = portValue) { ... }
```

R6: Excluir chave de autenticação do backup do Android

Adicione uma exclusão específica em `data_extraction_rules.xml` para impedir que a chave de autenticação seja incluída no backup:

```
<cloud-backup>
  <exclude domain="sharedpref" path="AppPrefs.xml"/>
</cloud-backup>
```

Prioridade 3 — Média

R7: Sanitizar respostas de StatusPages

Substitua `cause.toString()` por uma mensagem de erro genérica para evitar vazamento de nomes de classes de exceção:

```
call.respondText("Internal server error", status =
    HttpStatusCode.InternalServerError)
```

R8: Adicionar Configuração CORS

Instale o Ktor Plugin CORS para rejeitar solicitações de origem cruzada de fontes não-LAN:

```
install(CORS) { allowHost("localhost"); allowSameOrigin = true }
```

R9: Remover a permissão CHANGE_NETWORK_STATE

Essa permissão é declarada no `AndroidManifest.xml`, mas nunca é usada no código. Remova-a para reduzir a superfície de ataque declarada do aplicativo.

R10: Corrigir bug de registro do BroadcastReceiver

O registro do receptor em `onStart()` é protegido por `Build.VERSION.SDK_INT >= 0` e depois por `SDK_INT >= TIRAMISU`, o que significa que ele é registrado apenas na API 33+. Na API 26–32, `onStop()` chama `unregisterReceiver()` em um receptor nunca registrado, lançando uma exceção. Correção:

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    registerReceiver(serverStateReceiver, IntentFilter("SERVER_STATE_UPDATE"),
        RECEIVER_NOT_EXPORTED)
} else if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
    registerReceiver(serverStateReceiver, IntentFilter("SERVER_STATE_UPDATE"))
}
```

Priority 4 — Baixa

R11: Habilitar ProGuard/R8 em builds de lançamento

Definir `isMinifyEnabled = true` e configurar `proguard-rules.pro` para manter os alvos de reflexão do Ktor/Netty. Isso dificulta significativamente a engenharia reversa de APKs.

R12: Adicionar validação da força da chave de autenticação

Impor uma complexidade mínima de chave: pelo menos 8 caracteres, misturando letras, dígitos e símbolos. Exibir um indicador de força na caixa de diálogo.

R13: Expandir a cobertura de testes

O repositório contém apenas testes básicos (`ExampleUnitTest`, `ExampleInstrumentedTest`). Adicione testes unitários para a lógica de autenticação do servidor web, limitação de volume,

validação de porta e testes de integração para as rotas do Ktor usando o `TestApplicationEngine` do Ktor.

10. Lições Aprendidas

Esta avaliação proporcionou informações valiosas sobre como as decisões de arquitetura de software influenciam diretamente a segurança do aplicativo.

Embora o aplicativo de Controle Remoto de Volume tenha sido originalmente projetado para uso pessoal em redes domésticas confiáveis, revisitar o projeto sob a perspectiva de segurança revelou diversos riscos que não foram considerados durante o desenvolvimento inicial.

A lição mais significativa foi que as vulnerabilidades de segurança são frequentemente introduzidas por meio de escolhas da arquitetura de projeto, e não por bugs de implementação. Recursos como comunicação HTTP em texto não criptografado, autenticação por meio de parâmetros de URL e a ausência de limitação de taxa funcionaram corretamente do ponto de vista do desenvolvimento, mas introduziram uma superfície de ataque desnecessária quando avaliados sob a ótica da segurança.

A avaliação também reforçou a importância da modelagem de ameaças. Ao mapear fluxos de dados, limites de confiança e potenciais agentes de ameaça, tornou-se mais fácil identificar cenários de ataque realistas e priorizar os riscos de acordo com seu impacto e probabilidade.

Outra lição importante foi o valor da validação manual. Diversas descobertas inicialmente identificadas por meio de análise estática e revisão de arquitetura exigiram testes práticos para determinar se representavam preocupações de segurança reais. Esse processo ajudou a distinguir riscos teóricos de descobertas confirmadas.

Por fim, este projeto demonstrou os benefícios de revisar softwares previamente concluídos sob uma perspectiva profissional diferente. O que começou como um projeto de desenvolvimento para Android se tornou uma oportunidade de aplicar conceitos de segurança de aplicativos, metodologias de avaliação de riscos e técnicas de relatórios estruturados a um sistema do mundo real.

Além das próprias descobertas técnicas, esta avaliação demonstrou o valor de revisitar projetos concluídos anteriormente com uma mentalidade focada em segurança. A capacidade de avaliar, questionar e reavaliar decisões de design anteriores provou ser tão valiosa quanto o desenvolvimento do aplicativo original.

Essa experiência fortaleceu as habilidades de desenvolvimento de software e análise de segurança e destacou a importância de integrar considerações de segurança em todo o ciclo de vida do desenvolvimento de software.